COMP 138 RL: Homework 2: Reinforcement Learning in Racetrack Problem

Longji Li

March 5, 2025

1 Introduction

Monte Carlo methods can be used in solving daily problems, such as self-driving vehicles and game robots. This paper presents using Monte Carlo methods to solve the racetrack[1] problem.

2 Problem

According to the textbook, a racetrack problem is manipulating a race car to navigate through a curved track while gaining acceleration and avoiding hitting the boundary (crushing). The purpose of the task is driving the car to reach the finish line as quick as possible.

2.1 Environment

The maps in the experiment are two different tracks that one is a (32x16) grid map and the other is a (32x32) grid map, as shown in figure 1.

3 Related Work

The objective of on-policy Monte Carlo (MC) control is to learn an optimal policy π that maximizes the return value and minimizes the number of steps to reach the finish line. As shown in figure 2, the algorithm illustrates that the car will learn and improve the policy by using the $\epsilon - greedy$ policy itself to select the best action among the possible actions in each state, but, with ϵ probability, it will randomly select an action. In the additional question, we show the implementation of the off-policy MC control in the racetrack problem.

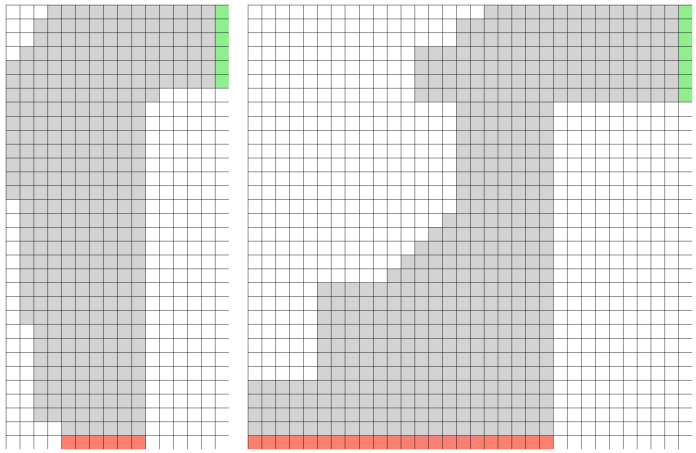

Figure 1: The track maps

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$
Initialize:
  $\pi \leftarrow$ an arbitrary ε-soft policy
  $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
  $Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):
  Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)
      For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Figure 2: On-policy MC control

4 Approach

4.1 State/ Action

Since we are solving the racetrack problem, the agent is a race car. The state of the car is a set $S(x, y, v_x, v_y)$ where x and y are the coordinates of the car, v_x and v_y are the velocity components.The maximum of the velocity is less than 5. The actions are a set of accelerations

$$[0,0], [1,0], [0,1], [1,1], [-1,0], [0,-1], [-1,1], [1,-1], [-1,-1]$$

4.2 Setup

We implement a line of red cells as the starting line, meaning the car will start from a random cell in this line. The green line is the finish line where the task will end once the car reaches any cell of it. The gray area is the track, and the white area is the boundary.

In each episode (driving task), we randomly select a starting cell. Then the car start the racetrack task on the track area. If the car runs out of the track (for example, hitting the white cells), it will return to a random cell in the starting line and assign a -1000 point as the penalized reward; otherwise, it explores the track and receives a -1 point reward iteratively until it finishes the task with +10 points (finishing reward).

4.3 Experiments

We conducted 6 trials using different finishing rewards, episodes, and maps (3 trials on the 32x16 grid map, 3 trials on the 32x32 grid map). For each model in the small map, we trained in 10000 episodes and each episode with 5000 max step size, which means that for each episode, if the car ran over 5000 steps, it would be sent back to a random start cell because we do not want the car to spend a long period of time on track. For the 3 models in the large map, we train 100000 episodes with the same max-step size.

4.4 32x16 grid map: finishing reward = 10, 100, 10000

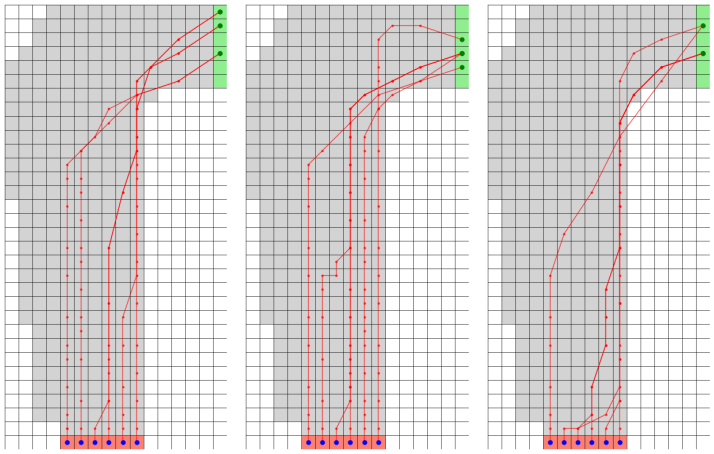

Figure 3: Simple track with different finishing rewards (left graph: +10, middle graph: +100, right graph: +10000)

We find that, in each trial, the trajectories show that the car successfully reached

the finish line. This result illustrates that based on MC control, the car can learn and improve the policy while exploring the map and finally generate the optimal policy that helps the car reach the finishing cells from each starting cell. In figure 4, it shows the rewards over episodes:

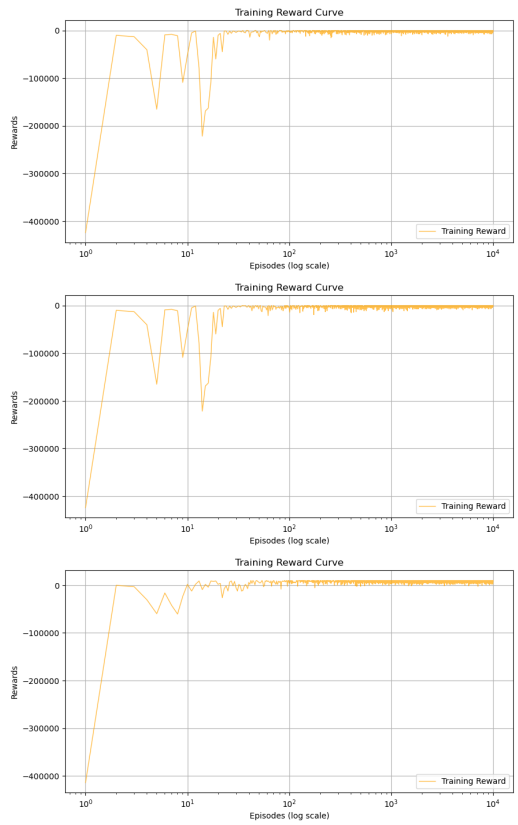

Figure 4: 32x16 grid map with different finishing rewards (left graph: +10, middle graph: +100, right graph: +10000)

This reward convergence illustrates that after 1000 iterations, the car can finish each task with a stable and high total reward.

4.5 32x32 grid map: finishing reward = 10, 100, 10000

For the 3 trials on the second map, the result, as shown in figure 5, is similar to the previous trials; the car also reached the finish line, and the total reward also converged after 1000 iterations (Figure 6).

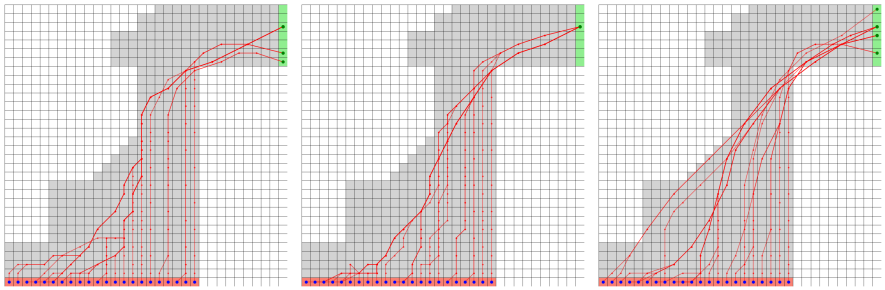

Figure 5: Complex track with different finishing rewards (left graph: +10, middle graph: +100, right graph: +10000)

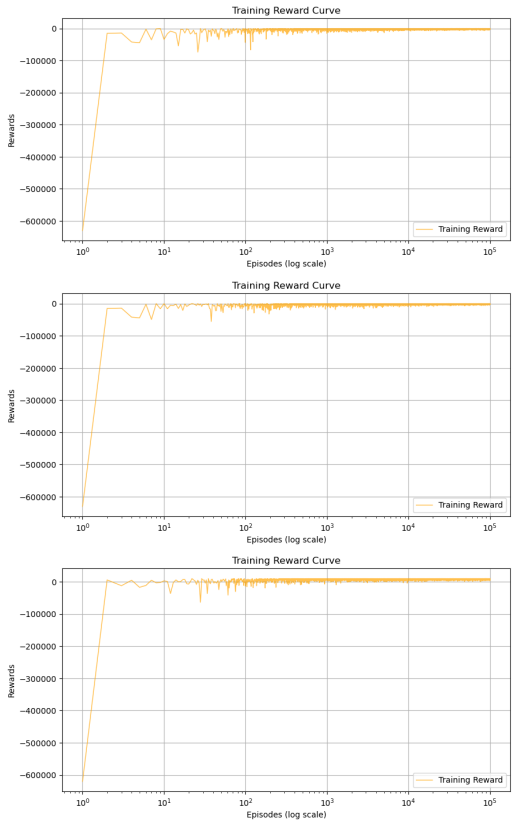

Figure 6: Complex track with different finishing rewards (top graph: +10, middle graph: +100, bottom graph: +10000)

5 Additional Question

To investigate the influence of off-policy MC control on the racetrack problem, we implemented the off-policy MC control (the algorithm used in the experiment

is the same as the one in the book). We trained the model in 10000 episodes using the max-step size of 5000. For the behavior policy, we implemented the biased policy and random-selection policy. Using the biased policy, the car can only select actions [-1,0], [-1,1], [0,1],[0,0], as we know where the finish line is, so we want the car to move forward to the upper-right corner and ignore other actions.

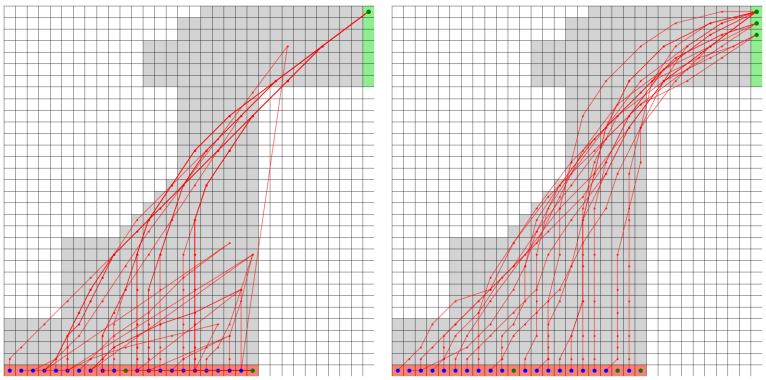

Figure 7: Left: car selects actions based on a biased behavior policy; right: car randomly selects actions.

However, the result shows that biased behavior policy generates a trajectory, which is a long path. On the contrary, when allowing the car to randomly select possible actions in each state, the trajectories are shorter than the first one.

6 Conclusion

This experiment explores the effectiveness of Monte Carlo (MC) control methods in solving the Racetrack problem, demonstrating how an agent (race car) can learn an optimal policy to reach the finish line as quick as possible. Through On-Policy MC Control, we find that the car progressively updates its policy using exploration, minimizing the number of steps required to complete the task. The results show that, regardless of different finishing rewards and map sizes, the policy successfully converges over time, leading to stable and improved performance.

Furthermore, we investigated the impact of Off-Policy MC Control, where the car learns an optimal policy using a behavior policy. The results show that when a biased behavior policy (always choose the actions that move up or right) is used, the final trajectories tend to be longer and suboptimal. However, when the behavior policy is random selection, the car can effectively find shorter paths to reach the finish line. This demonstrates the behavior policy plays an important role in updating policy.

Overall, this study confirms that both On-Policy and Off-Policy MC Control successfully help the agent find an optimal driving strategy with different parameter setups. Future work is needed when exploring larger maps with different maps and policies.

7 Extra Credit

7.1 Exercise 5.4

To change the equation to make it more efficient, we can change the $Q(S_t, A_t)$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}(G - Q(S_t, A_t))$$

where the $N(S_t, A_t)$ is the number of visits of the state and action pair.

7.2 Exercise 5.5

For this problem, we know that the nonterminal state can transit to the terminal state with a probability $1 - p$ and transit back to the nonterminal state with probability p. Since the first-visit MC method only considers the average of the returns based on first visits to the state, the first-visit method estimates the return

$$V = G = 1 + 1 + .. + 1 = 10$$

because it would measure the return from the time of the first visit until the end of the episode, which accumulates all rewards. Since the $\gamma = 1$, the V = 10. For the every-visit methods, we will calculate the v = average of the G over all visits.

$$V = \frac{1 + 2 + 3 + ... + 9 + 10}{10} = \frac{55}{10} = 5.5$$

So, V=5.5

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction.* Bradford Book, Cambridge, MA, second edition, 2018.